Master Informatique



Reimplementing `egrep`, supporting simplified Extended Regular Expressions

Sofija Granet, Jingru Zhao

October 2026

Academic year 2026/2027

Contents

1 Introduction **2**
- 1.1 Context of the project . . . 2
- 1.2 Regular expressions and the ERE standard . . . 2
- 1.3 Overview of the search process . . . 2
- 1.4 Data structures . . . 3

2 Algorithms from the Literature **4**
- 2.1 Transforming a RegEx into an automaton . . . 4
- 2.2 Determinization: the subset construction . . . 4
- 2.3 DFA minimization . . . 5
- 2.4 Searching a line with an automaton . . . 5
- 2.5 Exact substring search: Knuth-Morris-Pratt . . . 6

3 Experimental Setup : testbed and configuration **6**
- 3.1 Corpus construction . . . 6
- 3.2 Pattern selection . . . 7
- 3.3 Measurement protocol . . . 7
- 3.4 Correctness validation . . . 7

4 Performance evaluation : experiment results **8**
- 4.1 Overall command performance . . . 8
- 4.2 KMP versus the automaton baseline . . . 8
- 4.3 Controlled input-size scaling . . . 9
- 4.4 Automaton performance across different books . . . 10

5 Conclusion **11**

Note on the use of Artificial Intelligence **11**

Bibliography **12**

1 Introduction

1.1 Context of the project

Pattern matching in text is a classical problem in computer science, used in tools such as the UNIX commands `grep` and `egrep`, text editors, compilers (through lexical analysis) and, at a larger scale, the offline indexing of search engines. Formally, given a text file T composed of l lines and a pattern P expressed as a regular expression, the problem consists in determining, for every line of T, whether P occurs as a substring of that line. The output of our program is therefore the set of lines of T that contain at least one match of P, exactly as produced by `egrep`.

1.2 Regular expressions and the ERE standard

A *regular expression* (RegEx) is an algebraic representation of a language recognizable by a finite automaton, in the sense of formal language theory. The POSIX Extended Regular Expression (ERE) standard [1] defines a rich grammar, including character classes, repetition operators (`?`, `{n,m}`), and anchors (`^`, `$`). For the purpose of this project, we restrict ourselves to the following operators: **concatenation** of symbols (implicit operator), **alternation** (`|`), **Kleene closure**, or *star* (`*`), **Kleene plus** (`+`), i.e. one or more repetitions, expanded by the parser as $R^+ = RR^*$, **grouping** via parentheses `( )`, the **universal character** (`.`), matching any *single* character, **literal ASCII letters**.

Any RegEx built from these constructs denotes a regular language, recognizable by some finite automaton. This equivalence is the theoretical basis of our implementation: rather than interpreting the regular expression character by character, we transform it once into an automaton, and then use that automaton to test membership efficiently on every line of the input text.

1.3 Overview of the search process

Searching a text for an arbitrary ERE pattern involves the following steps:

1. **Parsing**: the pattern is parsed into an abstract syntax tree (AST) that respects operator precedence: star binds tightest, then concatenation, then alternation.

2. **Automaton construction**: the AST is translated into a non-deterministic finite automaton with ϵ-transitions (NDFA), following Aho and Ullman [2]. The subset construction turns it into an equivalent DFA, which is then minimized to obtain the smallest DFA recognizing the same language.

3. **Matching**: each line is scanned once from left to right, tracking the set of active DFA states. A new candidate starts at every position and all candidates advance together, which amounts to running the DFA on every suffix of the line and gives `egrep`'s substring semantics. When only line selection is needed, the scan stops at the first accepting state. A nullable pattern matches every line, including the empty one.

Substring search with KMP When the pattern is only a sequence of characters without any operators, the problem is reduced to substring search, for which we also implement the Knuth-Morris-Pratt algorithm [3] as a faster alternative to the steps described above.

1.4 Data structures

Each stage uses the representation best suited to its dominant operation:

- **Syntax tree**: All nodes use the same `RegExTree` class: an integer `root`, which encodes either an operator (`CONCAT`, `ALTERN`, `STAR`, `DOT`, `EMPTY`) or a character literal, and an `ArrayList<RegExTree>` of children. The `+` quantifier is expanded by the parser into concatenation and star ($R^+ = RR^*$).
- **NDFA**: States are integer indices, and each state owns a list of outgoing `Edge(symbol, target)` records (`List<List<Edge>>`). Symbol `-1` denotes an ϵ-transition, `-2` the universal character, and `0`–`52` the input-symbol classes. Per-state edge lists give direct $O(1)$ access to a state's successors, which is what ϵ-closures and the subset construction need.
- **DFA** (before and after minimization): A complete, dense `int[][]` transition table with 53 columns, a `boolean[]` of accepting states, and a start state. The columns are the 52 ASCII letters plus a single class shared by all other code points: since only letters can appear as literals, all other characters are indistinguishable to the automaton. Transitions are looked up by direct array indexing; for each input character, the matcher performs one lookup per active DFA state, so the cost per character depends on the number of active states.
- **Auxiliary structures**: The subset construction maps each set of NDFA states (`BitSet`) to a DFA state through a `Map<BitSet, Integer>`; the matcher tracks active DFA states with two `BitSet`s swapped at each character.
- **KMP**: An immutable pattern string and an integer array `lps`. For each position i, `lps[i]` stores the length of the longest proper prefix of `pattern[0..i]` that is also a suffix of that substring. The table is constructed once and reused for each line. During matching, a single integer tracks the number of pattern characters currently matched.

2 Algorithms from the Literature

Throughout this section, m represents the size of the pattern (number of symbols and operators), n the length of a line, N the number of NDFA states, $|Q|$ the number of DFA states, and k the number of input-symbol classes ($k = 53$ in our implementation).

2.1 Transforming a RegEx into an automaton

Kleene's theorem [4] states that regular expressions and finite automata describe the same class of languages. Its constructive direction is what makes automaton-based search possible.

Problem Given a regular expression R of size m, build an NDFA A such that $L(A) = L(R)$.

Thompson's construction The method presented by Aho and Ullman [2] is Thompson's construction [5]. It proceeds by induction on the syntax tree: each sub-expression produces an automaton fragment with a single entry state and a single exit state.

The resulting automaton has at most $2m$ states, every state has at most two outgoing transitions, and the single accepting state has none. The time complexity of the construction is in $O(m)$. Thompson himself did not determinize the automaton: he simulated it directly by maintaining the set of active states. Since the automaton has at most $2m$ states, each step costs $O(m)$, so a line of length n is processed in $O(mn)$ time.

Alternative in the literature Glushkov's position automaton [6] avoids ϵ-transitions by having exactly one state per letter occurrence in the pattern and an initial state, but may have a quadratic number of transitions, making it quite dense. Thompson's construction is simpler to implement inductively, which is why we use it here.

2.2 Determinization: the subset construction

Problem Given an NDFA with N states, build a DFA recognizing the same language.

Subset construction Rabin and Scott [7] showed that every NDFA has an equivalent DFA whose states are sets of NDFA states. The *ϵ-closure* of a set S is the set of states reachable from S through ϵ-transitions only. The initial DFA state is the ϵ-closure of the NDFA's initial state, and the transition on symbol a from a DFA state S is the ϵ-closure of the states reachable from S by one a-transition. A worklist algorithm builds only the subsets reachable from the initial state; a DFA state is accepting if its subset contains an accepting NDFA state.

Complexity The DFA may have up to 2^N states, and some languages really do need that many: recognising words whose j-th letter from the end is a takes $j+1$ NDFA states but at least 2^j DFA states, since the automaton must remember the last j letters read [8]. Building each DFA state costs $O(kN)$, so the construction is exponential in the worst case, though rarely so on hand-written patterns.

2.3 DFA minimization

Problem Given a complete DFA, find an equivalent DFA with the minimum number of states.

Theoretical basis By the Myhill-Nerode theorem [9], every regular language has a unique minimal complete DFA, up to renaming of states [8]. It is obtained by removing unreachable states and merging *equivalent* states, where $p \equiv q$ if, for every word w, reading w from p ends in an accepting state exactly when reading w from q does. Since the subset construction only builds reachable states, merging equivalent states is sufficient in our setting.

Moore's algorithm Moore [10] computes $\equiv$ by partition refinement. The initial partition is $\{F, Q \setminus F\}$, where F is the set of accepting states. At each round, two states stay in the same block only if, for every symbol, their successors lie in the same block. The process stops when a round leaves the partition unchanged. Each round increases the number of blocks, so there are at most $|Q| - 2$ refining rounds, each costing $O(k|Q|)$, for a total of $O(k|Q|^2)$. This is the algorithm we use in our implementation.

Alternative in the literature Hopcroft's algorithm [11] refines the partition with a worklist of splitters and, when a block is split, only enqueues the smaller part. This yields $O(k|Q| \log |Q|)$, the best known worst-case bound. Brzozowski's algorithm [12] reverses and determinizes the automaton twice; it is very short to implement but exponential in the worst case, since it relies on the subset construction.

2.4 Searching a line with an automaton

Problem Given a DFA A recognizing $L(R)$ and a line $w = w_1 \dots w_n$, decide whether some factor $w_i \dots w_j$ belongs to $L(R)$. Many strategies are possible, including

1. **Restart at every position.** Run A from each position i until it accepts or reaches the dead state. This costs $O(n^2)$ in the worst case.

2. **Automaton for $\Sigma^* R$.** Determinize $\Sigma^* R$ (equivalently `.*R`) instead of R: a single run then reads each character once, in $O(n)$. However, the prefix Σ^* can make determinization exponential. For $R = a(a|b)^{j-1}$, the DFA of R has $O(j)$ states, whereas $\Sigma^* R$ is exactly the language of words whose j-th letter from the end is a, which requires 2^j states.

3. **Parallel simulation.** Maintain the set S_t of DFA states reached after reading $w_1 \dots w_t$ by all runs started so far: $S_{t+1} = \{\delta(q, w_{t+1}) : q \in S_t \cup \{q_0\}\}$. Since runs that reach the same state are merged, $|S_t| \leq |Q|$, and the cost is $O(n \cdot |Q|)$ per line. This is the strategy we are using in our implementation.

Backtracking engines Many widely used regex libraries (Python, `java.util.regex`) do not build automata but explore the pattern by backtracking. This supports features beyond regular languages, such as backreferences, but leads to exponential running time on some patterns, whereas automaton-based search is linear in n for a fixed pattern [13].

2.5 Exact substring search: Knuth-Morris-Pratt

Problem Given a pattern $P = p_1 \dots p_m$ made of literal characters (no operators) and a line w of length n, decide whether P occurs as a factor of w. The naive algorithm, which compares P at every position, costs $O(mn)$ in the worst case.

The KMP algorithm Knuth, Morris and Pratt [3] precompute a *failure function* π: for $1 \leq q \leq m$, $\pi[q]$ is the length of the longest proper prefix of $p_1 \dots p_q$ that is also a suffix of it. During the search, when a mismatch (or failure) occurs after q characters have been matched, the algorithm continues with $q \leftarrow \pi[q]$ instead of moving back in the text: the characters already read are never re-examined. Preprocessing takes $O(m)$ time and the search $O(n)$, with at most $2n$ character comparisons.

Link with automata KMP is a compact representation of the minimal DFA of $\Sigma^* P$, which has $m + 1$ states: the failure function replaces its $O(mk)$ transition table by an array of size $O(m)$. For a literal pattern, the previously-mentioned strategy 2 applies without any blow-up, which is why KMP can outperform the general pipeline: a single active state, and no subset construction or minimization.

Alternatives in the literature The Boyer-Moore algorithm [14] compares the pattern from right to left and can skip characters of the text. Its running time is sublinear on average when the alphabet is large compared with the pattern.

3 Experimental Setup : testbed and configuration

Configuration The experiments were conducted on a machine with an Apple M5 processor and 16GB of RAM, running macOS 26.5. The algorithms and tests were implemented using Java 21 and Python 3.9.6.

Note on `egrep` Our reference is GNU grep (version 3.12), installed through Homebrew: the default `grep` in macOS is the BSD implementation, which differs from GNU grep in both behaviour and performance. We invoke it as `grep -E` rather than `egrep` because the two are the same command, but recent versions of GNU grep print a deprecation warning for `egrep`, which would pollute the output our tests compare.

3.1 Corpus construction

To evaluate our implementation on real texts rather than randomly generated ones, we built our test corpus from the Project Gutenberg digital library [15]. Project Gutenberg provides access to public-domain books, which offers two important properties for a representative testbed ; **natural text statistics** (unlike randomly generated strings, these texts follow the statistical distribution of natural language such as word length, letter frequency, and repetition of common substrings, which is representative of the kind of input the `egrep` command is realistically used on) and **file size** (books range from a few hundred kilobytes to several megabytes, so we can vary the input size n and observe how running time scales).

We downloaded 11 text files from Project Gutenberg's top 100 most downloaded books, with sizes ranging from about 170 KB to 5.4 MB.

Note on end of line characters Some files in the Gutenberg Project have Windows-style end of line characters (\r\n) ; we have normalized the files (LF line endings, no BOM) so that every testbed has the same plain line structure and all programs treat \r as an ordinary character.

3.2 Pattern selection

To produce informative graphs, we made different sets of patterns, along the following axes:

- **plain literal patterns** of varying length (short vs long substrings) and varying frequency of occurrence in the corpus (rare vs common substrings)
- **patterns with alternation**
- **patterns with star and period** since they involve automata with a lot of active states
- **adversarial patterns**, chosen to stress a specific weakness: patterns causing a large blow-up in the subset construction, such as `(a|b)*a(a|b)(a|b)(a|b)(a|b)(a|b)`
- and the self-repetitive pattern `aaaaab` used in Section 4.3, which is the worst case for naive substring search and therefore the clearest illustration of KMP's advantage.

3.3 Measurement protocol

Two protocols are used ;

- **Whole-program timings** measure the complete process, from starting the program to producing the output, JVM start-up included. This is the only protocol that can include `grep`, which cannot report its internals. Each measurement is the mean of 10 runs after three discarded warm-up runs, so timings are warm-cache. All programs write their output to a real file, never `/dev/null`, since GNU `grep` detects it and stops at the first match.
- **In-memory timings** measure matching alone, on text already loaded in memory. Each case uses three fresh JVMs; each JVM warms both engines for at least five rounds and one second, then records 20 rounds, alternating the engine order. JVM start-up, construction, file I/O and output are excluded, and no timing is ever obtained by subtraction.

3.4 Correctness validation

Before measuring performance, we validated correctness by comparing the output of our program against the output of the system's `grep -E` command on every (pattern, file) pair in our test set. This is done by the `conformity.py` python script.

4 Performance evaluation : experiment results

4.1 Overall command performance

In this experiment, we compare the total running time of GNU grep, our automaton engine and KMP using the same input file, `pg100.txt` (The Complete Works of William Shakespeare, 5.44 MB). The measurements include the entire process, from starting the program to producing the final output.

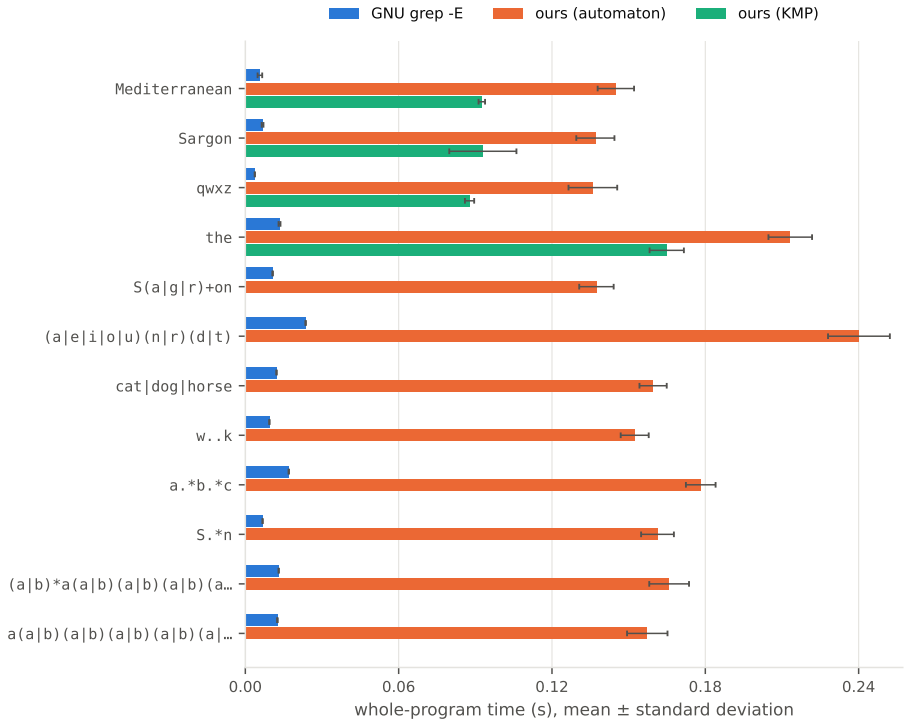


Figure 1: Total running time on `pg100.txt`, from program startup to completion. Each bar shows the average of 10 runs, with error bars showing one standard deviation. KMP is tested only on literal patterns.

Figure 1 shows that GNU grep is the fastest tool for all tested patterns. KMP is faster than our automaton for the four literal patterns, but remains slower than `grep`. It is only tested on literal patterns because it does not support regular-expression operators. These timings cover the complete programs, so the differences reflect startup, construction, file reading and output as well as matching.

4.2 KMP versus the automaton baseline

In this experiment, we compare our KMP implementation with our automaton engine using four literal patterns. Both are tested on the same input file, `pg100.txt` (5.44 MB).

Figure 2 shows that KMP is faster than our automaton engine for all four literal patterns. The difference is especially clear in the right-hand plot, where KMP takes less than 10 ms, compared to around 40–55 ms for the automaton.

KMP is also faster in the left-hand plot, but the difference is less noticeable. This is because the total running time includes other steps, such as starting the program, building the automaton, reading the file, and producing the output. As a result, even

though KMP is much faster at matching, the improvement in the overall running time is smaller.

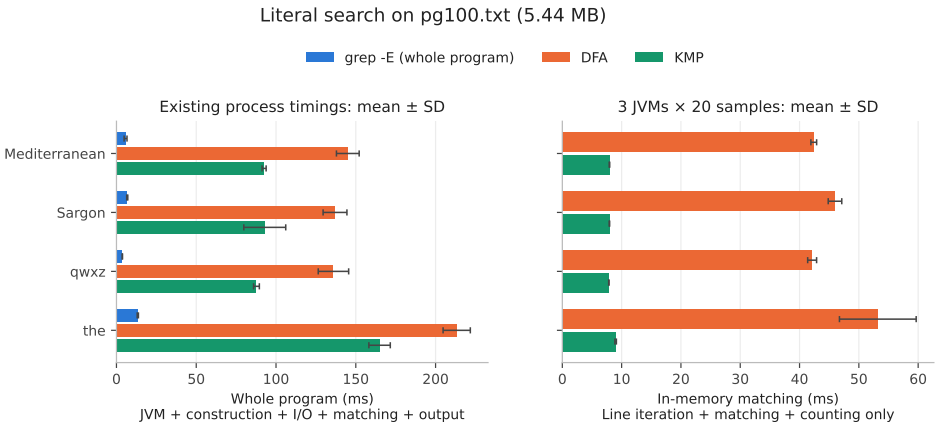


Figure 2: Performance comparison between KMP and our automaton engine on `pg100.txt`. The left plot shows the total execution time, while the right plot shows the matching time. Error bars represent one standard deviation.

4.3 Controlled input-size scaling

In this experiment, we test how the running time of KMP and our DFA matcher changes with the input size. We use the same pattern, `aaaaab`, on files of 1, 2, 4 and 8 MiB. Each file contains only the letter `a`, divided into lines of 4096 characters. Since the pattern ends with `b`, no matches are found.

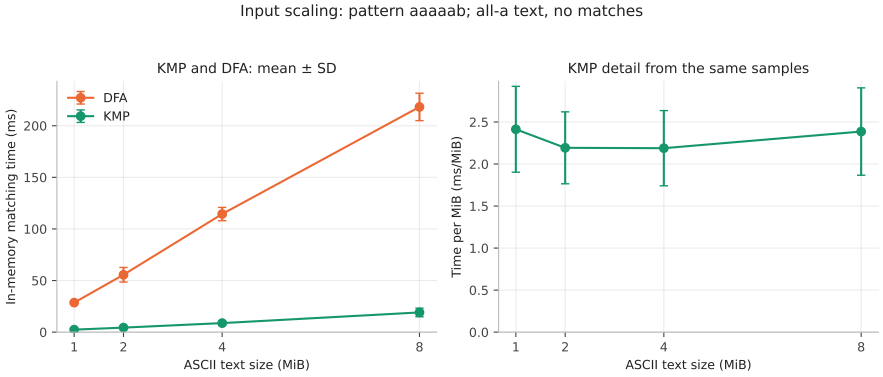


Figure 3: Search time for the fixed pattern `aaaaab` on 1, 2, 4 and 8 MiB of text containing only the letter `a`, divided into 4096-character lines. No matches occur. The left plot compares the average in-memory search time of both engines, while the right plot shows KMP's average time per MiB. Each point is based on 20 samples from each of three JVMs, and error bars show one standard deviation. Startup, construction, file I/O and output are excluded.

Figure 3 shows that both algorithms take more time as the input gets larger. The lines are almost straight, but the DFA matcher takes much longer as the file size increases. For the largest input, the DFA matcher takes about 200 ms, while KMP takes around 20 ms.

The right-hand plot shows that KMP's average time per MiB stays between about 2 and 2.5 ms, with only small differences between the tested file sizes. This suggests that KMP's running time increases roughly in proportion to the amount of text being searched, which is what we would expect for a fixed pattern. However, this experiment only uses repetitive text with no matches. Therefore, the performance difference we observe here may not be the same for other types of input.

4.4 Automaton performance across different books

In this experiment, we test how our automaton engine performs on 11 Gutenberg books of different sizes. We use four search patterns to see how the running time changes as the files get larger.

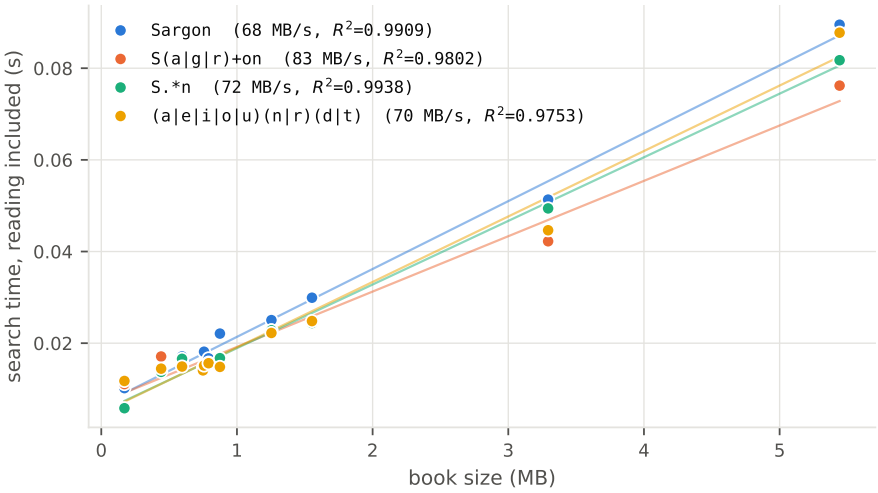


Figure 4: Search time of our automaton engine on 11 Gutenberg books of different sizes. Each point shows the median of 10 measurements for one pattern and one book. The lines show linear fits. The measurements include file reading and matching, but exclude startup, construction and output.

Figure 4 shows that search time increases as the books get larger. For all four patterns, the results are close to straight lines, suggesting roughly linear growth, which is consistent with the predicted $O(n)$ complexity. Although the patterns have different running times, they follow a similar trend. Since the books differ in content and line length, these results may not apply to all types of input.

Two internal measurements complete this picture. First, minimization has a real effect on patterns with alternation, reducing the DFA of `(a|e|i|o|u)(n|r)(d|t)` from 11 states to 5 and that of `a(a|b)(a|b)(a|b)(a|b)(a|b)(a|b)(a|b)` from 17 to 10, while literal patterns are already minimal; across our pattern set the minimal DFAs range from 4 to 65 states, and building them accounts for between 0.3% and 1.1% of the search time. Second, although the parallel simulation costs $O(n \cdot |Q|)$ in theory, the number of simultaneously active states never exceeded 4 on our corpus, with an average between 1.97 and 2.59 per character, even for the pattern whose minimal DFA has 65 states. The worst case is therefore far from being reached on natural text, and the cost per character behaves as a small constant.

5 Conclusion

In this project, we have implemented a clone of `egrep` for a simplified subset of extended regular expressions, following the full pipeline from a syntax tree to an ϵ-NDFA, a deterministic automaton obtained by the subset construction, and a minimal automaton obtained by Moore's partition refinement, together with a Knuth-Morris-Pratt engine for the special case of purely literal patterns. Correctness was established before timing: on every pattern of our test set and every book of the corpus, both engines print exactly the lines that `grep -E` prints, with the same exit status.

The experiments show that GNU grep remains largely faster than our implementation on every pattern tested, and that KMP clearly outperforms the automaton on literal patterns. This is the expected outcome, since we know that `egrep` is finely optimized.

First, the running time of our program varies far less across patterns than `grep`'s does: `egrep` is very fast on a rare literal, which it can scan by skipping blocks of text, and much slower on a pattern of nested alternations, where it can no longer skip, whereas our cost per character is almost independent from the pattern.

Second, the size of the automaton does not explain the running time. Our minimal DFAs range from 4 to 65 states, yet the largest of them is not the slowest to run: the pattern `(a|b)*a(a|b)(a|b)(a|b)(a|b)(a|b)`, whose minimal DFA has 65 states, is searched faster than `a.*b.*c`, whose minimal DFA has 5. Building the automaton, minimization included, accounts for between 0.3% and 1.1% of the total time, so the quadratic bound of Moore's algorithm never becomes a concern at this scale.

The gap with `egrep` therefore comes less from the search strategy than from everything around it: starting a JVM, reading the file, decoding UTF-8 and splitting lines. The `egrep` command works on bytes, and our implementation decodes code points.

Perspectives Several directions would narrow that gap. Working on bytes rather than code points, and matching on a byte-level automaton, would remove the decoding cost at the price of a more complex treatment of multi-byte characters.

Building DFA states only when the text reaches them would make the exponential blow-up of the subset construction harmless and allow the simpler $\Sigma^* R$ strategy to be used for every pattern instead of the parallel simulation we adopted.

Furthermore, Boyer-Moore would complement KMP by skipping characters rather than merely avoiding backtracking.

Note on the use of Artificial Intelligence During this project, we have sometimes used Large Language Models (LLM) to help us in our work ; Anthropic's *Claude* and OpenAI's *ChatGPT*. We have used it primarily to :

- help us make sense of certain algorithms and methods
- generate comments in our code to make it clearer to read
- generate python code to download books, and for our experiments.

Bibliography

[1] *The Open Group Base Specifications Issue 7: Extended Regular Expressions (ERE).* The Open Group / IEEE, 2018. URL `http://pubs.opengroup.org/onlinepubs/7908799/xbd/re.html`.

[2] Alfred V. Aho and Jeffrey D. Ullman. *Foundations of Computer Science*. Computer Science Press, New York, NY, 1992.

[3] Donald E. Knuth, James H. Morris, and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977. doi: 10.1137/0206024.

[4] Stephen C. Kleene. Representation of events in nerve nets and finite automata. In Claude E. Shannon and John McCarthy, editors, *Automata Studies*, number 34 in Annals of Mathematics Studies, pages 3–41. Princeton University Press, 1956.

[5] Ken Thompson. Programming techniques: Regular expression search algorithm. *Communications of the ACM*, 11(6):419–422, 1968.

[6] Victor M. Glushkov. The abstract theory of automata. *Russian Mathematical Surveys*, 16(5):1–53, 1961.

[7] Michael O. Rabin and Dana Scott. Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2):114–125, 1959.

[8] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation.* Addison-Wesley, 3rd edition, 2006.

[9] Anil Nerode. Linear automaton transformations. *Proceedings of the American Mathematical Society*, 9(4):541–544, 1958.

[10] Edward F. Moore. Gedanken-experiments on sequential machines. In Claude E. Shannon and John McCarthy, editors, *Automata Studies*, number 34 in Annals of Mathematics Studies, pages 129–153. Princeton University Press, 1956.

[11] John Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. In Zvi Kohavi and Azaria Paz, editors, *Theory of Machines and Computations*, pages 189–196. Academic Press, 1971.

[12] Janusz A. Brzozowski. Canonical regular expressions and minimal state graphs for definite events. In *Proceedings of the Symposium on Mathematical Theory of Automata*, volume 12 of *MRI Symposia Series*, pages 529–561. Polytechnic Press of the Polytechnic Institute of Brooklyn, 1962.

[13] Russ Cox. Regular expression matching can be simple and fast. `https://swtch.com/~rsc/regexp/regexp1.html`, 2007.

[14] Robert S. Boyer and J. Strother Moore. A fast string searching algorithm. *Communications of the ACM*, 20(10):762–772, 1977.

[15] Project Gutenberg. Project gutenberg: Free ebooks. `http://www.gutenberg.org/`.